

Software Requirements Specification (SRS)

EN2H Booking Platform Backend API

1. Introduction

1.1 Purpose

The purpose of this document is to outline the software requirements for the EN2H Booking Platform backend API. This system serves as the foundational backend infrastructure for a service-based business, enabling staff to manage service catalogs and allowing customers to book those services.

1.2 Scope

The system is a RESTful API built with NestJS, TypeScript, and PostgreSQL. It handles authentication, service catalog management, and booking workflows. The system ensures robust data integrity through transactional guards, concurrent booking prevention, and proper status transition rules. It does not include a frontend interface but provides a complete OpenAPI (Swagger) documentation layer for client integrations.

2. Overall Description

2.1 User Characteristics

There are two primary actors interacting with the system:

- **Staff/Administrators (Authenticated):** Can create and manage services, view all bookings, and update the status of existing bookings (e.g., from PENDING to COMPLETED).
- **Customers (Public/Unauthenticated):** Can browse active services and create bookings for available time slots.

2.2 Operating Environment

- **Runtime:** Node.js (v18+)
 - **Framework:** NestJS (v10)
 - **Database:** PostgreSQL (v15+)
 - **ORM:** Prisma
 - **Deployment:** Docker (containerized) or Vercel (Serverless Functions)
-

3. System Features & Functional Requirements

3.1 Authentication Module (*/auth*)

- **REQ-AUTH-1:** The system shall allow staff to register with an email and password.
- **REQ-AUTH-2:** The system shall hash passwords using `bcrypt` before persistence.
- **REQ-AUTH-3:** The system shall issue a JWT Access Token and Refresh Token upon successful login.
- **REQ-AUTH-4:** The system shall store a hashed version of the refresh token in the database to mitigate token theft.
- **REQ-AUTH-5:** The system shall allow users to obtain a new access token using a valid refresh token.
- **REQ-AUTH-6:** The system shall allow users to log out, which invalidates their current refresh token in the database.

3.2 Services Module (*/services*)

- **REQ-SRV-1:** The system shall allow authenticated staff to create a new service with a title, description, duration (minutes), and price.
- **REQ-SRV-2:** The system shall allow the public to retrieve a list of all *active* services, supporting pagination and text search.
- **REQ-SRV-3:** The system shall allow authenticated staff to update service details.
- **REQ-SRV-4:** The system shall implement a "soft delete" mechanism. When a service is deleted, its `isActive` flag is set to false to preserve booking history integrity.

3.3 Bookings Module (*/bookings*)

- **REQ-BKG-1:** The system shall allow public users to create a booking for a specific service, providing their name, email, phone, date, and time.
 - **REQ-BKG-2:** The system shall prevent double-booking. The combination of `serviceId`, `bookingDate`, and `bookingTime` must be globally unique.
 - **REQ-BKG-3:** The system shall reject booking dates in the past.
 - **REQ-BKG-4:** The system shall reject bookings for services that do not exist or are inactive.
 - **REQ-BKG-5:** The system shall allow authenticated staff to view all bookings, with support for filtering and pagination.
 - **REQ-BKG-6:** The system shall allow authenticated staff to update the status of a booking (e.g., `PENDING` -> `CONFIRMED` -> `COMPLETED`).
 - **REQ-BKG-7:** The system shall enforce strict status transition rules (e.g., a `CANCELLED` booking cannot be transitioned to `COMPLETED`).
-

4. Non-Functional Requirements

4.1 Security

- All sensitive endpoints must be protected by JWT-based Bearer authentication.
- Passwords and Refresh Tokens must never be stored in plain text.
- CORS must be enabled to allow frontend client integration.

4.2 Data Integrity

- The database schema must enforce relational constraints (Foreign Keys).
- Money values must be stored using the `DECIMAL(10, 2)` data type to prevent floating-point rounding errors.
- Unique constraints must be enforced at the database level to prevent race conditions during concurrent booking requests.

4.3 Documentation & Usability

- The system must auto-generate OpenAPI (Swagger) documentation available at the `/docs` endpoint.
- The Swagger UI must persist authorization tokens during the session.

4.4 Deployment & Scalability

- The system must be fully containerized using Docker, providing a `docker-compose.yml` file for easy local provisioning.
- The application must be stateless (excluding the database) to allow serverless deployment on platforms like Vercel.

5. Database Schema (Entity Relationship)

- **User:** `id`, `name`, `email` (unique), `password`, `refreshToken`, `createdAt`, `updatedAt`
- **Service:** `id`, `title`, `description`, `duration`, `price`, `isActive`, `createdById` (FK to User), `createdAt`, `updatedAt`
- **Booking:** `id`, `customerName`, `customerEmail`, `customerPhone`, `serviceId` (FK to Service), `bookingDate`, `bookingTime`, `status`, `notes`, `createdAt`, `updatedAt`
 - *Constraint:* `@@unique([serviceId, bookingDate, bookingTime])`